

Day 4, Session 2: Remote Objects, RPC & Communicating Sequential Processes

Duration: 2 hours (theory + live demonstrations)

Prerequisites: Days 1, 2, & 3 (OOP in Python, Networking, Sockets)

Demos folder: [day4/demo/](#)

Table of Contents

1. [The Remote Procedure Call \(RPC\) Paradigm](#)
2. [Remote Method Invocation with Pyro4](#)
 - [What is Pyro4?](#)
 - [The Name Server and Pyro Daemon](#)
 - [Developing a Client-Server Application with Pyro4](#)
 - [Chaining Remote Objects](#)
3. [Transparent RPC with RPyC](#)
 - [Symmetric RPC Architecture](#)
 - [Exposing Services with RPyC](#)

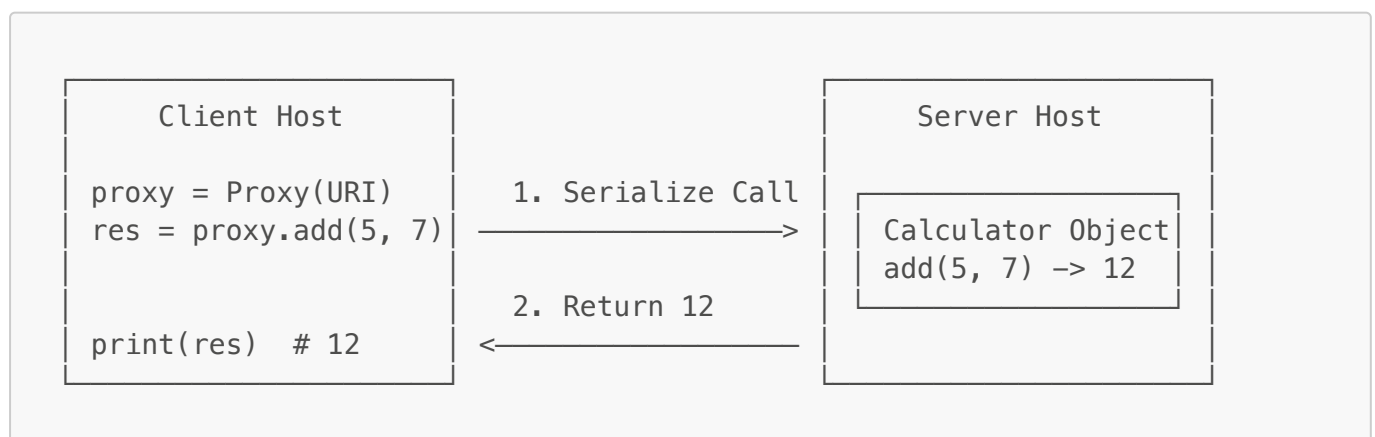
1. The Remote Procedure Call (RPC) Paradigm

In standard Python code, when you instantiate an object and call a method:

```
calc = Calculator()
res = calc.add(10, 20)
```

The method executes locally within the same process.

In **Distributed Systems**, we often want that object to live on a **remote machine** with specialized hardware (e.g. 512 GB RAM, high-end database, or GPUs), while the client calls methods **as if the object were local**:



The underlying network socket communication, serialization, and deserialization are handled automatically by the RPC framework.

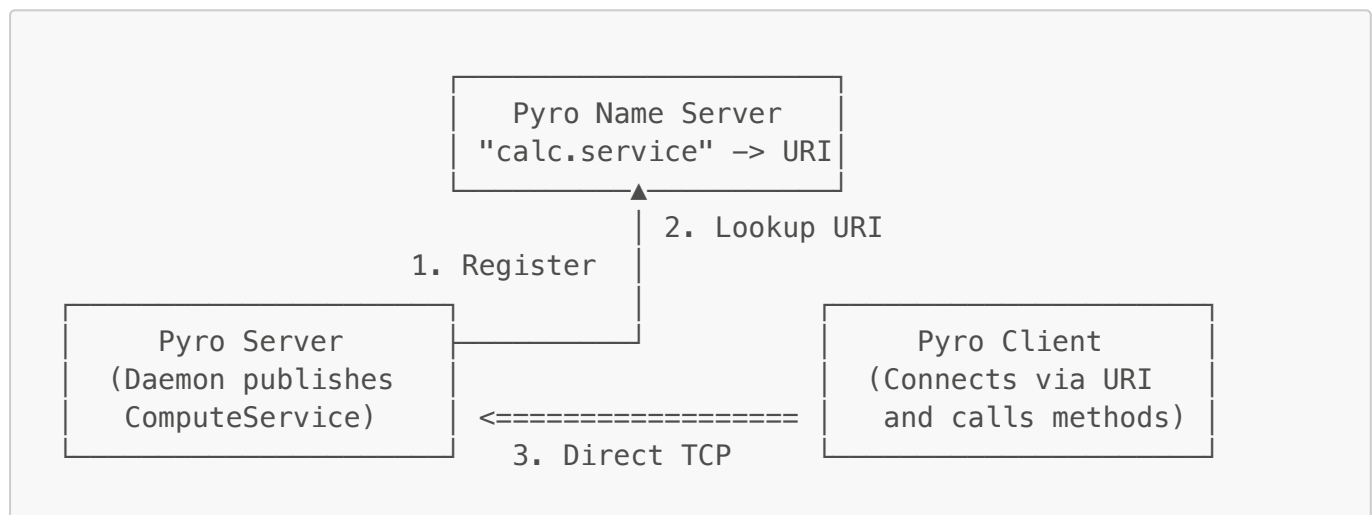
2. Remote Method Invocation with Pyro4

Pyro4 stands for **Python Remote Objects**:

- It allows you to expose ordinary Python classes over the network.
- It feels just like normal Python code (object-oriented RPC / RMI).
- Supports automatic network serialization, exception propagation, and security tokens.

The Name Server and Pyro Daemon

To connect clients to servers without hardcoding IP addresses, Pyro uses a **Name Server**:



1. Start the Name Server:

```
pyro4-ns
```

2. **Server registers** its service under a human-friendly name ("**calc.service**").
3. **Client queries Name Server** to locate the current IP/Port and connects directly.

Developing a Client-Server Application with Pyro4

1. The Server (**server.py**)

```
import Pyro4

@Pyro4.expose # Security: explicitly allow remote method calls
class CalculatorService:
    def add(self, a, b):
        return a + b
```

```
def power(self, base, exp):
    return base ** exp

daemon = Pyro4.Daemon()           # Start the network listener
ns = Pyro4.locateNS()             # Locate the Pyro Name Server
uri = daemon.register(CalculatorService) # Register the class
ns.register("service.calc", uri)   # Publish name to Name Server

print("Calculator service is running and ready for clients...")
daemon.requestLoop()
```

2. The Client (`client.py`)

```
import Pyro4

# Look up the remote object via the Name Server
calc = Pyro4.Proxy("PYRONAME:service.calc")

# Transparent remote method call!
result = calc.power(2, 10)
print("Result from remote server:", result) # 1024
```

Chaining Remote Objects

In distributed microservices, objects often interact with other remote services:

- **Service A (Auth)** verifies user identity.
- **Service B (Analytics)** queries the database.
- **Service C (Notifier)** sends notifications.

With Pyro4, remote proxies can be created inside another remote object. Service A can seamlessly forward data or chain calls to Service B and Service C without exposing internal cluster topology to the end client.

Demo Time

Now, let's see these theoretical topics in action.

- `03_pyro4_remote_objects.py` —
 1. `demo_remote_method_invocation()` — Exposing remote objects, proxy calls, and stats
 2. `demo_pyro_chaining()` — Chaining remote service objects across distributed microservices

3. Transparent RPC with RPyC

While Pyro4 focuses on object-oriented remote services, **RPyC** (Remote Python Call) provides **transparent, symmetrical RPC**:

- It allows you to directly manipulate remote variables, remote modules, and remote objects as if they were local.
- If an exception occurs on the remote server, RPyC reconstructs the full remote stack trace on your client!

```
import rpyc

# Connect to remote RPyC server
conn = rpyc.connect("192.168.1.50", 18812)

# Directly execute operations on the remote server's Python interpreter:
print("Remote OS PID:", conn.modules.os.getpid())
remote_val = conn.modules.math.sqrt(144)
print("Evaluated remotely:", remote_val) # 12.0
```

Exposing Services with RPyC

For secure, controlled access, RPyC uses **Service** classes. Any method prefixed with **exposed_** is made available to clients:

```
import rpyc

class MyService(rpyc.Service):
    def exposed_multiply(self, a, b):
        return a * b

# Client calls: conn.root.multiply(5, 6) -> 30
```

Session 2 Summary & What's Next

Framework	Paradigm	Key Concept	Use Case
Pyro4	Object-Oriented RMI	@Pyro4.expose, Name Server	Calling methods on remote Python objects
RPyC	Transparent RPC	rpyc.connect(), exposed_	Remote execution, remote scripting, testing

What's Coming Tomorrow (Day 5)

Tomorrow is the grand finale: **GPU Programming with Python!**

- GPU hardware architecture: thousands of parallel ALUs, streaming multiprocessors.

- **PyCUDA:** Writing CUDA kernels in Python, GPU memory allocation, and grid/block threads hierarchy.
 - **Numba / NumbaPro:** JIT compilation and `@cuda.jit` decorators.
 - **PyOpenCL:** Cross-vendor GPU acceleration (NVIDIA, AMD, Intel, Apple Silicon).
-

End of Day 4